DOCUMENTO MAESTRO DE ORGANIZACIÓN & ARQUITECTURA

Resolución Metaheurística del FHCP Challenge | Equipo de 5 Integrantes

Proyecto:	Resolución Metaheurística del Problema del Ciclo Hamiltoniano (FHCP Challenge)
Metodología:	Aula Invertida y Desarrollo Ágil de Software de IA
Dirección:	Dirección de Proyectos de Ingeniería de Software e Investigación en IA
Estándar:	Strict Code Interface, GitHub Flow & Overleaf LaTeX IEEE/APA

1. Entorno de Trabajo y Herramientas Oficiales (Stack Tecnológico)

Para garantizar la reproducibilidad científica, la mantenibilidad del código y la calidad del informe técnico, el equipo trabajará bajo las siguientes plataformas y estándares oficiales:

Dimensión	Plataforma / Herramienta	Estándar Operativo y Reglas de Integración
Control de Versiones	GitHub	Ramas: **main** (protegida, solo código probado), **develop** (integración) y ramas *feature/** por algoritmo. Ningún código entra a develop sin pasar el validador estricto de ciclos.
Lenguaje y Librerías	Python 3.10+ / NumPy, pandas, matplotlib, seaborn	NumPy para operaciones matriciales rápidas de grafos; pandas y seaborn para benchmarking. **Prohibido usar librerías con metaheurísticas pre-programadas**; implementación desde cero.
Redacción de Reporte	Overleaf (LaTeX) + Zotero	Estructura modular con archivos *.tex* independientes por sección y gestión de referencias en formato IEEE/APA mediante *references.bib*.
Gestión de Proyecto	GitHub Projects (Kanban)	Tablero Kanban dividido en las 4 fases oficiales: Investigación, Core/Dataset, Implementación y Benchmarking/Reporte.

2. Arquitectura del Repositorio y Contrato de Interfaz

Se establece una estructura de repositorio modular y aislada para garantizar que el motor de pruebas (*benchmark_runner.py*) pueda ejecutar de manera transparente cualquiera de los 6 algoritmos desarrollados.

Estructura Oficial del Repositorio GitHub:

```
fhcp-metaheuristics/
■■■ .github/workflows/          # CI/CD pipelines y validadores automáticos
■■■ data/fhcp/                  # Dataset oficial del FHCP Challenge Set (archivos .hcp)
■■■ docs/latex/                 # Código fuente LaTeX modular para Overleaf
■■■ results/                    # CSVs de ejecuciones estadísticas y gráficas
■■■ src/
■   ■■■ core/                    # Parser .hcp, Grafo en memoria, Fitness, Validador
■   ■■■ utils/                   # Random Keys, Operadores combinatorios (2-opt, swaps), Métricas
■   ■■■ algorithms/              # Implementaciones de SA, GA, ACO, ABC, DE, PSO
```

```
■   ■■■ benchmark_runner.py     # Motor principal de pruebas automatizadas
■■■ tests/                      # Pruebas unitarias para validar grafos y componentes
```

Definición Formal del Contrato de Interfaz (BaseMetaheuristic):

```
from abc import ABC, abstractmethod
from typing import Dict, Any, List, Tuple
import numpy as np

class BaseMetaheuristic(ABC):
    """Interfaz abstracta obligatoria para todas las metaheurísticas."""
    def __init__(self, graph_data: Any, max_evals: int, timeout_sec: float, seed: int):
        self.graph = graph_data
        self.max_evals = max_evals
        self.timeout_sec = timeout_sec
        self.seed = seed
        np.random.seed(seed)

    @abstractmethod
    def solve(self) -> Dict[str, Any]:
        """
        Retorna obligatoriamente:
        - 'solution': List[int], permutación de longitud n sin repeticiones.
        - 'is_hamiltonian': bool, True si cierra un ciclo hamiltoniano válido.
        - 'max_fitness': float, número de aristas reales válidas.
        - 'iterations': int, número total de iteraciones.
        - 'evaluations': int, total de evaluaciones consumidas.
        - 'execution_time_sec': float, tiempo de ejecución en segundos.
        - 'convergence_history': List[Tuple[int, float]], [(eval_count, best_fitness)].
        """
        pass
```

3. Asignación Equilibrada de Roles para los 5 Integrantes

Para asegurar un equilibrio estricto de carga técnica y de investigación, cada integrante asumirá un componente de infraestructura core, una o dos metaheurísticas específicas y un bloque temático en Overleaf:

Integrante	Rol Principal	Infraestructura / Apoyo	Algoritmo(s)	Reporte LaTeX (Overleaf)
Integrante 1	Arquitecto Core & Grafo	Parser .hcp, Grafo en memoria, Fitness y Validador Estricto	Recocido Simulado (SA)	Teoría de Grafos, Ciclo Hamiltoniano, Mapeo y SA
Integrante 2	Líder de Benchmarking	Motor benchmark_runner.py y Exportación de Métricas	Algoritmo Genético (GA - Baseline)	Módulo Evolutivo, Métricas IA, Estándares y GA
Integrante 3	Especialista en Enjambres	Heurística Local de Visibilidad (Grados de Nodos)	Ant Colony Optimization (ACO)	Fundamentación ACO y Refuerzo de Feromonas en FHCP
Integrante 4	Especialista en Búsqueda Local	Módulo de Operadores Combinatorios (operators.py)	Artificial Bee Colony (ABC)	Fundamentación ABC y Benchmarking contra FHCP

Integrante	Rol Principal	Infraestructura / Apoyo	Algoritmo(s)	Reporte LaTeX (Overleaf)
Integrante 5	Especialista Continua-Discreta	Decodificador Random Keys (argsort vectores)	Evolución Diferencial (DE) y PSO	Fundamentación DE/PSO, Discretización y Conclusiones

Desglose de Entregables por Fases (1 a 4):

• Integrante 1 (Arquitecto Core + SA)

– **Fase 1:** Investigación teórica de Teoría de Grafos y definición formal de Ciclo Hamiltoniano NP-duro.

– **Fase 2:** Programación del parser de archivos *.hcp*, grafo en memoria, función de fitness con penalizaciones y Validador Estricto de Ciclos.

– **Fase 3:** Implementación de Simulated Annealing con operadores de intercambio combinatorio (swaps, 2-opt) y esquema de enfriamiento lento.

– **Fase 4:** Redacción en Overleaf del Marco Teórico de Grafos, definición del problema, función de costo y sección de SA.

• Integrante 2 (Líder de Benchmarking + GA)

– **Fase 1:** Investigación de métricas de desempeño algorítmico, Precisión, Recall, F1-Score y estándares del FHCP Challenge.

– **Fase 2:** Construcción del motor de pruebas *benchmark_runner.py* para ejecución automatizada y generación de reportes CSV/gráficas.

– **Fase 3:** Implementación del Algoritmo Genético (GA) como baseline, con cruza específica para permutaciones (OX o PMX) e inversión.

– **Fase 4:** Redacción del Módulo Evolutivo, métricas de IA, diseño del GA y compilación final de Tablas/Gráficas Comparativas.

• Integrante 3 (Especialista en Enjambres - ACO)

– **Fase 1:** Investigación de modelos de enjambre e inteligencia colectiva aplicados a optimización combinatoria.

– **Fase 2:** Diseño de la heurística de visibilidad local basada en grados de vértices para resolver cuellos de botella y nodos puente.

– **Fase 3:** Implementación de Ant Colony Optimization (ACO) con matriz de feromonas, regla de transición probabilística y evaporación.

– **Fase 4:** Redacción del marco teórico de ACO y análisis del refuerzo de feromonas en grafos difíciles.

• Integrante 4 (Especialista en Búsqueda Local - ABC)

– **Fase 1:** Investigación de optimización por colonia de abejas (Artificial Bee Colony) y búsqueda local distribuida.

– **Fase 2:** Creación del módulo *operators.py* con operadores de vecindad combinatoria (2-opt, inserción, swaps) compartidos.

– **Fase 3:** Implementación de ABC para permutaciones (abejas empleadas, observadoras con reclutamiento y exploradoras).

– **Fase 4:** Redacción del marco teórico de ABC y capítulo de Benchmarking contra los estándares del FHCP Challenge.

• Integrante 5 (Especialista Continua-Discreta - DE/PSO)

– **Fase 1:** Investigación de técnicas de adaptación de algoritmos continuos a espacios combinatorios discretos.

– **Fase 2:** Desarrollo del decodificador continuo-a-discreto *Random Keys* (argsort sobre vectores reales) en *encoding.py*.

– **Fase 3:** Implementación de Evolución Diferencial (DE) y Particle Swarm Optimization (PSO) operando sobre el decodificador.

– **Fase 4:** Redacción de la teoría de DE/PSO, explicación del mapeo de espacios continuos y Conclusiones Generales.

4. Protocolo Experimental y Reglas de Rigor Científico

Para garantizar la validez metodológica e impedir cualquier sesgo en las métricas comparativas, el equipo seguirá de manera estricta el siguiente protocolo experimental:

a) Selección Escalonada del Dataset FHCP:

• *Instancias Pequeñas ($n \leq 50$):* Depuración, CI/CD y validación rápida de convergencia.

• *Instancias Medianas ($50 < n \leq 500$):* Ajuste de hiperparámetros y benchmarking comparativo.

• *Instancias Grandes/Difíciles ($n > 500$ o trampas estructurales):* Evaluación de escalabilidad y evasión de mínimos locales.

b) Estandarización de Hardware:

La corrida definitiva del benchmark de los 6 algoritmos se ejecutará exclusivamente en una única máquina física designada, garantizando que la métrica de tiempo (*execution_time_sec*) sea 100% comparable y justa.

c) Ejecuciones Estadísticas y Reproducibilidad:

Debido al carácter estocástico de las metaheurísticas, cada algoritmo se ejecutará un mínimo de **N = 30 corridas independientes** con semillas predefinidas ($seed \in \{1001, ..., 1030\}$). Se reportará tasa de éxito (%), media, desviación estándar, mínimo y máximo.

d) Aterrizaje Matemático de Precisión, Recall y F1-Score:

Formulación de Métricas en IA: Métricas Micro / Local (Recuperación de Aristas por Solución):
• **TP (Verdaderos Positivos):** Número de aristas de la solución que existen en el grafo real G=(V,E).
• **FP (Falsos Positivos):** Aristas de la solución inexistentes en el grafo real (FP = n - TP).
• **FN (Falsos Negativos):** Aristas del ciclo hamiltoniano no incluidas (FN = n - TP).
• **Precisión Micro:** P = TP / n | **Recall Micro:** R = TP / n | **F1-Score Micro:** F1 = TP / n.
Un F1-Score Micro de 1.0 indica un ciclo hamiltoniano 100% válido y cerrado.

Métricas Macro / Global (Batería de Pruebas):
Evaluación de la tasa de éxito global (TP_macro = ejecuciones con F1_micro = 1.0) sobre el FHCP Challenge Set.

5. Flujo de Trabajo, Dependencias y Validación Cruzada

Cronograma de Dependencias y Ruta Crítica:

```
[Semanas 1-2] Fase 1: Investigación (Aula Invertida - Todos)
     ■
[Semanas 2-3] Fase 2: Infraestructura Core & Dataset
     ■■■> Bloqueante 1: Integrante 1 (parser .hcp, Graph, Fitness)
     ■■■> Bloqueante 2: Integrante 5 (decodificador Random Keys)
     ■■■> DESBLOQUEO TEMPRANO: Grafo Dummy (10 nodos) desde el Día 1
     ■
[Semanas 4-6] Fase 3: Implementación de Metaheurísticas (SA, GA, ACO, ABC, DE, PSO)
     ■
[Semanas 7-8] Fase 4: Benchmarking, Evaluación y Documentación en Overleaf
```

Protocolo de Code Review Cruzado y Antifraude:

• **Aprobación Cruzada Obligatoria:** Ningún Pull Request se fusionará a *develop* sin la aprobación formal de al menos 2 integrantes (incluyendo al Arquitecto Core o Líder de Benchmarking).

• **Validación Automática Incorrompible:** Cada PR ejecutará un script de prueba (*tests/test_validator.py*) que verificará que la solución sea una permutación de longitud n sin repetidos y con aristas válidas.

• **Protección contra Falsos Positivos:** La variable *is_hamiltonian* solo podrá ser modificada a True por el método *validate()* oficial. Prohibido asignarla manualmente en las metaheurísticas.

> **Oficialización del Proyecto:** El presente Documento Maestro constituye la guía oficial de arquitectura y gestión para el desarrollo del proyecto FHCP Challenge, asegurando el cumplimiento total de los requisitos de la práctica y de las directrices de la asignatura.